

# 0.26.1. En diagonale : deplace — de base, ailleurs

La diagonale du 0.26 à partir de (10, 0), avec le B.

## 1. Ce qu'il faut comprendre

C'est le 0.26, avec une autre lettre, le **B** ( `ALPHABET[1]`, la 2e lettre), et une autre place : le **départ (10, 0)** : la diagonale finit en (15, 5).

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.26.2 met les deux ensemble.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1787 octets** de cartouche.

### 1. UNE VARIABLE — X

```
uint8_t x = 10;    // la colonne du B
```

### 2. UNE VARIABLE — Y

```
uint8_t y = 0;    // sa ligne
```

### 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace(x, y, ALPHABET[1], 5, 5);

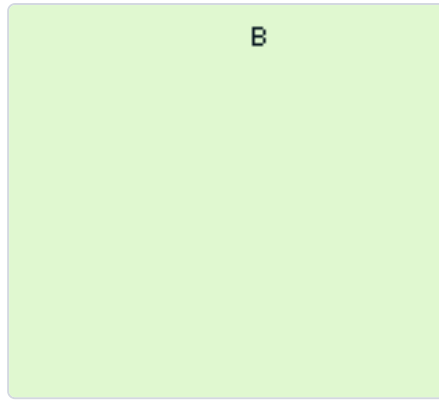
    while (true) {
        image();
    }
}
```

## 3. Ce qui se passe, étape par étape

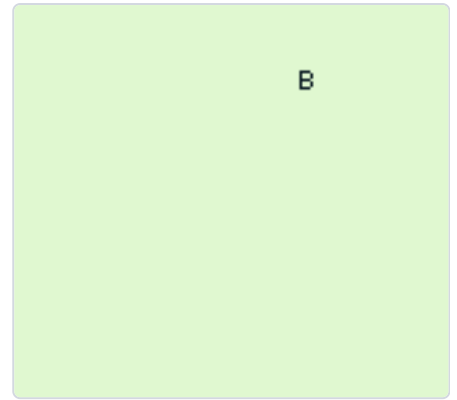
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



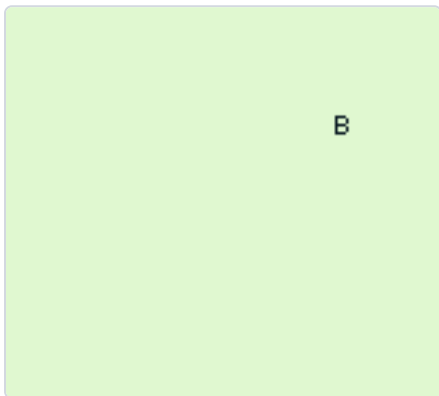
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

## 4. Ce qu'on doit voir

Un B qui descend en biais de (10, 0) à (15, 5).

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B passe par (12, 2)

✓ le B finit en (15, 5) (15,5)

Ensuite : **0.26.2. En diagonale : deplace — doublé, deux positions** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++